

Question No:3

Question Description:

There are n cities and m flight connections between them. Your task is to add new flights so that it will be possible to travel from any city to any other city. What is the minimum number of new flights required?

Input

The first input line has two integers n and m : the number of cities and flights. The cities are numbered $1, 2, \dots, n$.

After this, there are m lines describing the flights. Each line has two integers a and b : there is a flight from city a to city b . All flights are one-way flights.

Output

First print an integer k : the required number of new flights. After this, print k lines describing the new flights. You can print any valid solution.

Constraints

- $1 \leq n \leq 10^5$
- $1 \leq m \leq 2 \cdot 10^5$
- $1 \leq a, b \leq n$

✓ Logical Test Cases

Test Case 1

INPUT (STDIN)

```
4 5
1 2
2 3
3 1
1 4
3 4
```

EXPECTED OUTPUT

```
1
4 1
```

Test Case 2

INPUT (STDIN)

```
4 5
1 3
2 1
1 2
2 4
1 4
```

EXPECTED OUTPUT

```
2
4 3
3 1
```

Code:

```
1  #include <stdio.h>
2  #include <string.h>
3
4  #define MAXN 100005
5  #define MAXM 200005
6
7  /* ----- Graph structure ----- */
8
9  typedef struct Edge
10 {
11     int to;
12     int next;
13 } Edge;
14
15 Edge graph[MAXM * 2];
16 Edge reverseGraph[MAXM * 2];
17 Edge componentGraph[MAXM * 2];
18
19 int head[MAXN];
20 int reverseHead[MAXN];
21 int componentHead[MAXN];
22
23 int edgeCount = 0;
24 int reverseEdgeCount = 0;
25 int componentEdgeCount = 0;
26
27 /* ----- SCC data ----- */
```

```

28
29     int visited[MAXN];
30     int finishOrder[MAXN];
31     int orderCount = 0;
32
33     int component[MAXN];
34
35     /*
36      * Each SCC is represented by one vertex.
37      * We use the smallest/first vertex encountered as
38      * the component representative.
39      */
40     int componentCount = 0;
41
42     /* indegree of each SCC */
43     int indegree[MAXN];
44
45     /* ----- Add edges ----- */
46
47     void addEdge(Edge graph[], int head[], int *count, int u, int v)
48     {
49         graph[*count].to = v;
50         graph[*count].next = head[u];
51         head[u] = (*count)++;
52     }
53

```

```

55
56 void dfs1(int u)
57 {
58     visited[u] = 1;
59
60     for (int e = head[u]; e != -1; e = graph[e].next)
61     {
62         int v = graph[e].to;
63
64         if (!visited[v])
65             dfs1(v);
66     }
67
68     finishOrder[orderCount++] = u;
69 }
70
71 /* ----- Second DFS ----- */
72
73 void dfs2(int u, int root)
74 {
75     if (component[u] != -1)
76     {
77         /*
78          * If we reach another SCC, add an edge in
79          * the condensation graph.
80          */

```

```

80     */
81     if (component[u] != root)
82     {
83         addEdge(componentGraph,
84                 componentHead,
85                 &componentEdgeCount,
86                 component[u],
87                 root);
88
89         indegree[root]++;
90     }
91
92     return;
93 }
94
95 component[u] = root;
96
97 for (int e = reverseHead[u];
98      e != -1;
99      e = reverseGraph[e].next)
100 {
101     dfs2(reverseGraph[e].to, root);
102 }
103 }

```

```

105  /* ----- Find a sink reachable from SCC ----- */
106
107  int used[MAXN];
108
109  /*
110   * Returns a sink SCC reachable from u that has not
111   * already been used.
112   */
113  int findSink(int u)
114  {
115      if (used[u])
116          return -1;
117
118      used[u] = 1;
119
120      /*
121       * If this SCC has no outgoing edge, it is a sink.
122       */
123      if (componentHead[u] == -1)
124          return u;
125
126      for (int e = componentHead[u];
127           e != -1;
128           e = componentGraph[e].next)
129      {
130          int v = componentGraph[e].to;

```

```
131
132     int result = findSink(v);
133
134     if (result != -1)
135         return result;
136 }
137
138 return -1;
139 }
140
141 /* ----- Main ----- */
142
143 int main()
144 {
145     int n, m;
146
147     scanf("%d %d", &n, &m);
148
149     /* Initialize heads */
150     for (int i = 1; i <= n; i++)
151     {
152         head[i] = -1;
153         reverseHead[i] = -1;
154         component[i] = -1;
155     }
156 }
```

```

157      /*
158      * Read original graph and reverse graph.
159      */
160      for (int i = 0; i < m; i++)
161      {
162          int a, b;
163
164          scanf("%d %d", &a, &b);
165
166          addEdge(graph, head,
167                  &edgeCount, a, b);
168
169          addEdge(reverseGraph, reverseHead,
170                  &reverseEdgeCount, b, a);
171      }
172
173      /* ----- Kosaraju: first pass ----- */
174
175      for (int i = 1; i <= n; i++)
176      {
177          if (!visited[i])
178              dfs1(i);
179      }
180
181      /* Initialize component graph */
182      for (int i = 1; i <= n; i++)
183          componentHead[i] = -1;

```

```

184
185     /*
186     * ----- Kosaraju: second pass -----
187     *
188     * We use the vertex number itself as the SCC ID.
189     */
190     for (int i = orderCount - 1; i >= 0; i--)
191     {
192         int u = finishOrder[i];
193
194         if (component[u] == -1)
195         {
196             dfs2(u, u);
197             componentCount++;
198         }
199     }
200
201     /*
202     * If there is only one SCC, the graph is already
203     * strongly connected.
204     */
205     if (componentCount == 1)
206     {
207         printf("0\n");
208         return 0;
209     }
210
211     /* ----- Find source and sink SCCs ----- */

```

```

213     int sources[MAXN];
214     int sinks[MAXN];
215
216     int sourceCount = 0;
217     int sinkCount = 0;
218
219     for (int i = 1; i <= n; i++)
220     {
221         if (component[i] != i)
222             continue;
223
224         /*
225          * No incoming edges => source SCC.
226          */
227         if (indegree[i] == 0)
228             sources[sourceCount++] = i;
229
230         /*
231          * No outgoing edges => sink SCC.
232          */
233         if (componentHead[i] == -1)
234             sinks[sinkCount++] = i;
235     }
236
237     /*
238     * These arrays will contain the SCCs that we
239     * connect together.
240     */

```

```
241     int sourceList[MAXN];
242     int sinkList[MAXN];
243
244     int sourceListSize = 0;
245     int sinkListSize = 0;
246
247     /*
248     * First, for every source, try to find a unique
249     * reachable sink.
250     */
251     for (int i = 0; i < sourceCount; i++)
252     {
253         int s = sources[i];
254
255         int sink = findSink(s);
256
257         if (sink != -1)
258         {
259             sourceList[sourceListSize++] = s;
260             sinkList[sinkListSize++] = sink;
261         }
262     }
263
264     /*
265     * Add sources which were not included above.
266     */
267     for (int i = 0; i < sourceCount; i++)
268     {
```

```
269     int s = sources[i];
270
271     int found = 0;
272
273     for (int j = 0; j < sourceListSize; j++)
274     {
275         if (sourceList[j] == s)
276         {
277             found = 1;
278             break;
279         }
280     }
281
282     if (!found)
283         sourceList[sourceListSize++] = s;
284 }
285
286 /*
287  * Add sinks which were not included above.
288  */
289 for (int i = 0; i < sinkCount; i++)
290 {
291     int s = sinks[i];
292
293     int found = 0;
294
295     for (int j = 0; j < sinkListSize; j++)
```

```
296     {
297         if (sinkList[j] == s)
298         {
299             found = 1;
300             break;
301         }
302     }
303
304     if (!found)
305         sinkList[sinkListSize++] = s;
306 }
307
308 /*
309  * The above duplicate checking can be expensive.
310  * For n = 1e5, rebuild the lists using markers.
311  */
312
313 memset(used, 0, sizeof(used));
314
315 sourceListSize = 0;
316 sinkListSize = 0;
317
318 /*
319  * Pair every source with a reachable sink.
320  */
321 for (int i = 0; i < sourceCount; i++)
```

```
322     {
323         int s = sources[i];
324
325         memset(used, 0, sizeof(used));
326
327         int sink = findSink(s);
328
329         if (sink != -1)
330         {
331             sourceList[sourceListSize++] = s;
332             sinkList[sinkListSize++] = sink;
333         }
334     }
335
336     /*
337     * Add remaining sources.
338     */
339     int sourceMarked[MAXN] = {0};
340     int sinkMarked[MAXN] = {0};
341
342     for (int i = 0; i < sourceListSize; i++)
343         sourceMarked[sourceList[i]] = 1;
344
345     for (int i = 0; i < sinkListSize; i++)
346         sinkMarked[sinkList[i]] = 1;
347
348     for (int i = 0; i < sourceCount; i++)
```

```

349     {
350         if (!sourceMarked[sources[i]])
351         {
352             sourceList[sourceListSize++] = sources[i];
353             sourceMarked[sources[i]] = 1;
354         }
355     }
356
357     for (int i = 0; i < sinkCount; i++)
358     {
359         if (!sinkMarked[sinks[i]])
360         {
361             sinkList[sinkListSize++] = sinks[i];
362             sinkMarked[sinks[i]] = 1;
363         }
364     }
365
366     int answer = sourceListSize > sinkListSize
367                 ? sourceListSize
368                 : sinkListSize;
369
370     printf("%d\n", answer);
371
372     /*
373     * Connect the sink of one chain to the source
374     * of the next chain.
375     */

```

```
376     int common = sourceListSize < sinkListSize
377         ? sourceListSize
378         : sinkListSize;
379
380     for (int i = 0; i < common - 1; i++)
381     {
382         printf("%d %d\n",
383             sinkList[i],
384             sourceList[i + 1]);
385     }
386
387     /*
388     * Handle extra sources.
389     */
390     if (sourceListSize > sinkListSize)
391     {
392         int x = sinkList[common - 1];
393
394         for (int i = common; i < sourceListSize; i++)
395         {
396             printf("%d %d\n",
397                 x,
398                 sourceList[i]);
399
400             x = sourceList[i];
401         }
402     }
```

```

402
403     printf("%d %d\n",
404            x,
405            sourceList[0]);
406 }
407 /*
408  * Handle extra sinks.
409  */
410 else if (sinkListSize > sourceListSize)
411 {
412     int x = sinkList[common - 1];
413
414     for (int i = common; i < sinkListSize; i++)
415     {
416         printf("%d %d\n",
417                x,
418                sourceList[0]);
419
420         x = sinkList[i];
421     }
422
423     printf("%d %d\n",
424            x,
425            sourceList[0]);
426 }
427 else
428 {
429     /*
430     * Equal number of sources and sinks.
431     */
432     printf("%d %d\n",
433            sinkList[common - 1],
434            sourceList[0]);
435 }
436
437 return 0;
438 }

```

Output:

Test Case-1:

Output	
4	5
1	2
2	3
3	1
1	4
3	4
1	
4	1

Test Case-2:

Output	
4	5
1	3
2	1
1	2
2	4
1	4
2	
4	1
3	1

Keshav Kumar Rauniyar

CH.SC.U4CSE25236